LAB WRITEUP 04 - SNORT (TRYHACKME)

Room	Snort
Platform	TryHackMe - `https://tryhackme.com/room/snort`
Path	SOC Level 1 - Network Security / Snort module
Difficulty	Medium
Time to complete	About 2-3 hours
Tasks	11

> Note on figures: the numeric outputs below come from the room's own VM and ruleset (Snort build 149). Where a count depends on the ruleset version loaded, the command that produces it is given so the result can be reproduced on any build.

Objective

Learn what Snort is, how an IDS differs from an IPS, and how to run Snort in each of its four operation modes: sniffer, packet logger, NIDS/NIPS and PCAP investigation. The room then covers the Snort rule structure and writing custom rules to detect specific traffic.

Concepts covered

- IDS vs IPS, and the NIDS / HIDS / NIPS / HIPS / NBA family
- Snort operation modes and their command-line flags
- Reading binary Snort logs back with `-r`
- Snort rule anatomy: action, protocol, source, direction, destination, options
- General, payload and non-payload rule options

Task 1 - Introduction

No answer required. Snort is an open-source, rule-based Network Intrusion Detection and Prevention System, originally written by Martin Roesch and now maintained by Cisco Talos with the community. It can act as a packet sniffer (like `tcpdump`), a packet logger, or a full-blown NIDS/NIPS.

Task 2 - Interactive material and the VM

The attached VM holds a Task-Exercises folder on the Desktop with a `Config-Sample` folder and an `Exercise-Files` folder that contains a separate pcap/log/rules set per task. A `traffic-generator.sh` script generates the traffic needed by each exercise - and it is important to pick the exercise type that matches the task you are on.

Question	Answer	How
Output of `./easy.sh` from the Task-Exercises folder	Too Easy!	`cd Desktop/Task-Exercises` then run the script with `sudo ./easy.sh`; it prints the string.

Task 3 - Introduction to IDS/IPS

Question	Answer	Reasoning
Which type can help you stop the threats on a local machine?	HIPS	Host-based, and prevention means it blocks rather than only alerting.
Which type can help you detect threats on a local network?	NIDS	Network-based, detect-only: it watches traffic across the network and alerts.
Which type can help you detect the threats on a local machine?	HIDS	Host-based detection: monitors one endpoint and alerts.
Which type can help you stop the threats on a local network?	NIPS	Network-based prevention: it terminates malicious connections for the whole subnet.
Which described solution works by detecting anomalies in the network?	NBA	Network Behaviour Analysis learns normal traffic (baseline) and flags deviations, so it can catch behaviour with no matching signature.
According to its official description, what kind of NIPS is Snort?	full-blown	Snort's own description calls it "a full-blown network intrusion prevention system".
NBA's training period is also known as ...	baselining	The learning phase where normal traffic is profiled.

Detection techniques to remember: signature-based (matches known patterns), behaviour-based (anomaly/baseline), and policy-based (violations of a written policy).

Detection vs prevention in one sentence: an IDS alerts and waits for a human; an IPS alerts and acts, usually by dropping the packet or resetting the connection.

Task 4 - First interaction with Snort

Question	Answer	Command
Build number of the Snort instance	149	`sudo snort -V` - prints version and build information (flags are case-sensitive).
Number of rules loaded with `/etc/snort/snort.conf`	4151 (graded as "no answer needed" in the current room version)	`sudo snort -c /etc/snort/snort.conf -T` - the summary near the end of the output reports how many rules were loaded.
Number of rules loaded with `/etc/snort/snortv2.conf`	1	`sudo snort -c /etc/snort/snortv2.conf -T` - the second configuration file exists to demonstrate a minimal ruleset.

Flags learned: `-V` version, `-c` configuration file, `-T` test configuration and exit, `-q` quiet mode. Testing the configuration before running it is the habit that prevents a silent failure at 3 a.m.

Task 5 - Operation mode 1: Sniffer mode

No graded answers; the point is to compare the output of progressively more detailed flags while the traffic generator runs.

Flag	What it adds
`-v`	Verbose: prints TCP/IP header information, like `tcpdump`
`-d`	Adds the packet payload
`-e`	Adds link-layer (Ethernet/MAC) headers

Flag	What it adds
`-X`	Dumps the whole packet in hex and ASCII
`-i eth0`	Chooses the interface to listen on

Task 6 - Operation mode 2: Packet logger mode

Packets are written to disk instead of only to the screen. Logs are binary (tcpdump format) by default and can be read back with `-r`, or written as readable ASCII with `-K ASCII`.

Question	Answer	How
Source port used to connect to port 53 (from the `145.254.160.237` folder)	3009	Run `sudo snort -dev -K ASCII -l .`, generate the TASK-6 traffic, stop Snort, then look inside the ASCII log folder named after the remote IP. The file name shows the protocol and the two ports (`UDP:3009-53`), i.e. DNS from source port 3009.
IP ID of the 10th packet in `snort.log.1640048004`	49313	`snort -r snort.log.1640048004 -n 10` reads the binary log back and stops after ten packets; the ID field of the tenth packet header holds the value.
Referer of the 4th packet	`http://www.ethereal.com/development.html`	Add the hex/ASCII dump flag: `snort -r snort.log.1640048004 -n 10 -X`. The HTTP `Referer:` header is visible in the ASCII payload of the fourth packet.
Acknowledgement number of the 8th packet	`0x38AFFFF3`	Count down the packet headers in the same `snort -r` output.
Number of "TCP port 80" packets	41	`snort -r snort.log.1640048004 tcp port 80` - Snort accepts a BPF-style filter after the log file, so the count is simply the number of packets it processes.

Practical lesson: binary logs are compact and complete, but on Ubuntu the files are owned by root - use `sudo` or change ownership before reading them.

Task 7 - Operation mode 3: IDS/IPS

Snort is now pointed at the live interface with a full ruleset and generates alerts in real time.

Question	Answer	How
Number of HTTP GET methods detected	2	Run `sudo snort -c /etc/snort/snort.conf -A full -l .`, generate the TASK-7 traffic, stop, and read the summary: the HTTP Inspect section counts the GET methods seen.

`-A full` chooses the alert output mode; other options include `console`, `fast`, `cmg` and `none`. IPS mode additionally requires running inline so that Snort can drop the offending packets.

Task 8 - Operation mode 4: PCAP investigation

This is the day-to-day use for an analyst: replaying a captured file through rules.

Question	Answer	Command
Alerts generated by `mx-1.pcap` with the default configuration	170	`sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-1.pcap`
TCP segments queued	18	Same output, in the stream/statistics section
HTTP response headers extracted	4	Same output, in the HTTP Inspect section
Alerts generated by `mx-1.pcap` with the second configuration	68	`sudo snort -c /etc/snort/snortv2.conf -A full -l . -r mx-1.pcap`
Alerts generated by `mx-2.pcap` with the default configuration	340	`sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-2.pcap`
Number of detected TCP packets	82	Same output - the TCP port filter section of the statistics
Alerts generated by `mx-2.pcap` plus `mx-3.pcap`	1020	`sudo snort -c /etc/snort/snort.conf -A full -l . --pcap-list="mx-2.pcap mx-3.pcap"`

Why the counts differ between configurations: `snortv2.conf` loads a minimal ruleset, so far fewer rules can match. That difference is exactly how a SOC measures whether a ruleset change increased or reduced detection coverage.

Task 9 - Snort rule structure

The rule header is mandatory; the options in parentheses are where the real detection logic lives.

```
action  protocol  src_ip  src_port  direction  dst_ip  dst_port  (option; option; ...)
alert   tcp       any      any       ->         any     any       (msg:"SSH login"; sid:1000001; rev:1
;)
```

- Direction: `->` is one-way, `<>` is bidirectional. There is no `<-` operator.
- Actions: `alert` (IDS) and `reject` (IPS) are the two used in this room.
- General options: `msg`, `sid`, `rev`, `reference`, `gid`, `classtype`, `priority`, `metadata`.
- Payload options: `content`, `nocase`, `depth`, `offset`, `distance`, `within`, `pcre`, `byte_test`.
- Non-payload options: `flags`, `id`, `seq`, `ack`, `ttl`, `sameip`, `dsize`, `itype`, `icode`.

Local rules live in `/etc/snort/rules/local.rules`, editable with `sudo gedit /etc/snort/rules/local.rules`, and a rule is only used when the run command points at it (`-c local.rules`).

Question	Rule written	Answer
Filter IP ID `35369` in `task9.pcap` - what is the request name of the detected packet?	`alert ip any any <> any any (msg:"ID TEST"; id:35369; sid:1000001; rev:1;)`	TIMESTAMP REQUEST
Filter packets with the SYN flag - how many detected packets?	`alert tcp any any <> any any (msg:"FLAG TEST"; flags:S; sid:1000002; rev:1;)`	1
Filter packets with Push-Ack flags - how many detected packets?	`alert tcp any any <> any any (msg:"Push-Ack TEST"; flags:P,A; sid:1000003; rev:1;)`	216
Filter packets with the same source and destination IP - how many packets?	`alert ip any any <> any any (msg:"SAME-IP TEST"; sameip; sid:1000004; rev:1;)`	7

Question	Rule written	Answer
An analyst successfully modifies an existing rule - which option must be changed afterwards?	-	`rev`

Each rule was tested with `sudo snort -c local.rules -A full -l . -r task9.pcap`, the alert/log files were cleared between questions, and previously used rules were commented out with `#` before the next test so that old matches were not counted.

The `rev` convention: raising `rev` (revision) on every change keeps a history of what the rule looked like when an alert was generated, which matters enormously when a rule is tuned after a false positive.

Task 10 - Snort 2 operation logic: points to remember

No new tool use; this task consolidates the file layout and the run logic.

Item	Path / value
Main configuration file	`/etc/snort/snort.conf`
Minimal/alternative configuration used in the room	`/etc/snort/snortv2.conf`
Local rules file	`/etc/snort/rules/local.rules`
Ruleset directory	`/etc/snort/rules/`
Default log directory	`/var/log/snort` (`.` forces the current directory)
Default rules provided by the package	Deactivated in the room so exercises start from a clean slate
Configuration test	`sudo snort -c <conf> -T`
Read a saved log	`sudo snort -r <logfile>`

Snort 2 versus Snort 3 (worth knowing): Snort 3 replaces `snort.conf` with a Lua-based `snort.lua` plus `snort_defaults.lua`, which makes the configuration module-based. The rule syntax itself is largely compatible.

Task 11 - Conclusion

No answer required. The room finishes with the message that Snort is not only an IDS/IPS - it is also a sniffer, a packet logger and a PCAP analysis engine, and the same rule language drives all of them.

Key takeaways

1. IDS detects, IPS prevents - and the host/network split gives HIDS, HIPS, NIDS and NIPS.
2. Four operation modes: sniffer (`-v/-d/-e/-X`), logger (`-l`, `-K ASCII`), NIDS/NIPS (`-c ... -A ... -i ...`) and PCAP investigation (`-r file`).
3. `-T` always test the configuration before running it.
4. A Snort rule is header + options, and detection quality comes from choosing the right options - payload options for content, non-payload options for header-level behaviour such as `flags`, `id` and `sameip`.
5. Every rule change needs a `rev` increment, and every local rule needs a unique `sid`.
6. `--pcap-list` shows that Snort can process several capture files in one run, which is how a batch of captures is triaged in one pass.